

Configuration & Deployment Report: AWS Classic Load Balancer

Step-by-Step Hands-On Infrastructure Lab Guide

Topic:	AWS Elastic Load Balancing (Classic Load Balancer Architecture)
Status:	Successfully Deployed & Verified
Environment:	AWS Management Console (Region: us-east-1)

1. Theoretical Background

An **Elastic Load Balancer (ELB)** automatically routes incoming web and application layer traffic across multiple target endpoints (such as Amazon EC2 virtual servers, container environments, or specific IP endpoints). This orchestration significantly mitigates single points of failure, provides scalable elastic properties, and maintains uniform system performance under load spikes.

The **Classic Load Balancer (CLB)** represents the previous-generation load balancing paradigm within the AWS eco-system. It evaluates and processes traffic across multiple layers of the Open Systems Interconnection (OSI) communication model:

- **Layer 4 (Transport Layer):** Manages raw networking protocols such as TCP and SSL/TLS. It balances traffic at the packet level without inspecting payload contents.
- **Layer 7 (Application Layer):** Examines higher-level protocol headers to enable HTTP/HTTPS application features, facilitating capabilities such as session stickiness utilizing browser cookies.

Architectural Optimization Tip

While the Classic Load Balancer continues to remain operational for historic legacy environments and EC2-Classic frameworks, AWS explicitly recommends modern deployments utilize specialized **Application Load Balancers (ALB)** for refined URL path-based routing, or **Network Load Balancers (NLB)** for high-throughput, ultra-low latency Layer 4 routing matrices.

2. Step-by-Step Practical Implementation

This section provides an organized chronological blueprint detailing the provisioning, mapping, and orchestration steps taken within the AWS console to establish a highly available web architecture.

Step 1: Navigating to the EC2 Instances Dashboard

Access the AWS Management Console, establish connectivity to the **us-east-1 (N. Virginia)** region, and select the EC2 dashboard service node. The infrastructure pool initially shows zero running instances prior to starting provisioning operations.

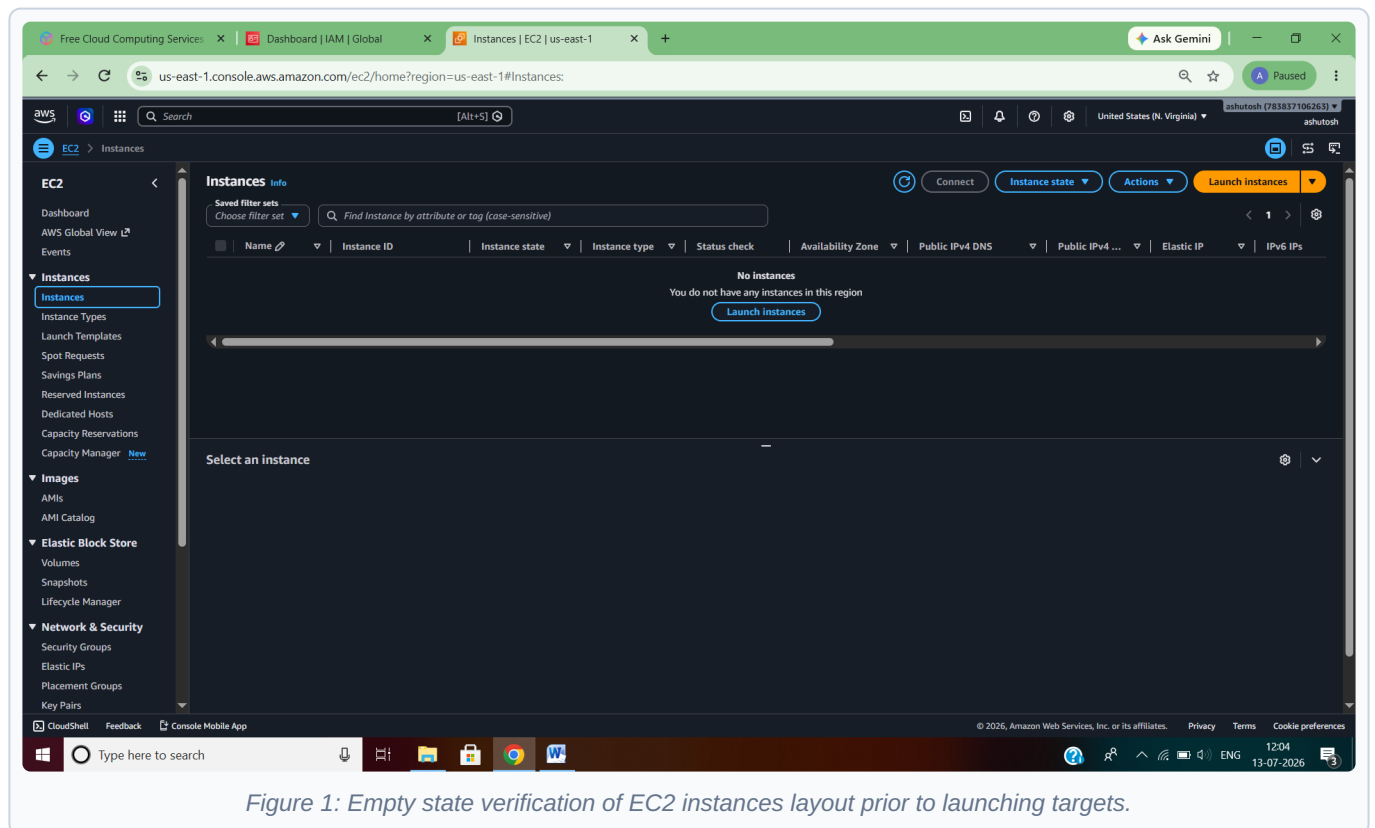
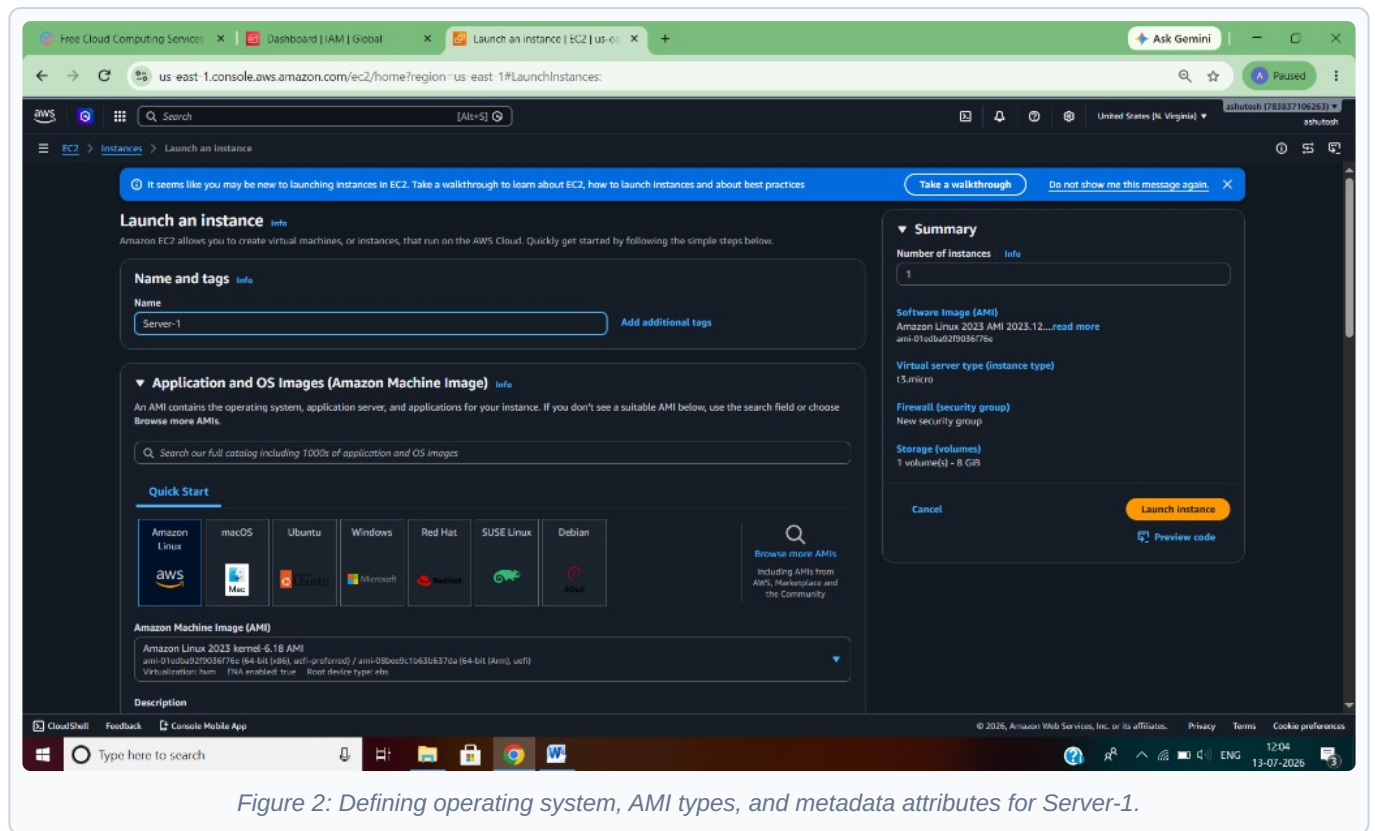


Figure 1: Empty state verification of EC2 instances layout prior to launching targets.

Step 2: Provisioning First Web Endpoint Backend (Server-1)

Initiate the instance configuration wizard. Provide the distinct identification name Server -1, select the cloud-optimized **Amazon Linux 2023 Amazon Machine Image (AMI)** running on a standard 64-bit architecture, and configure standard network subnets.



Step 3: Verification of Server-1 Launch Status

The wizard confirms the instantiation sequence. Instance ID *i-070e2088342250fc6* enters the initialization state successfully. Next steps options include setting alarms or preparing load balancers.

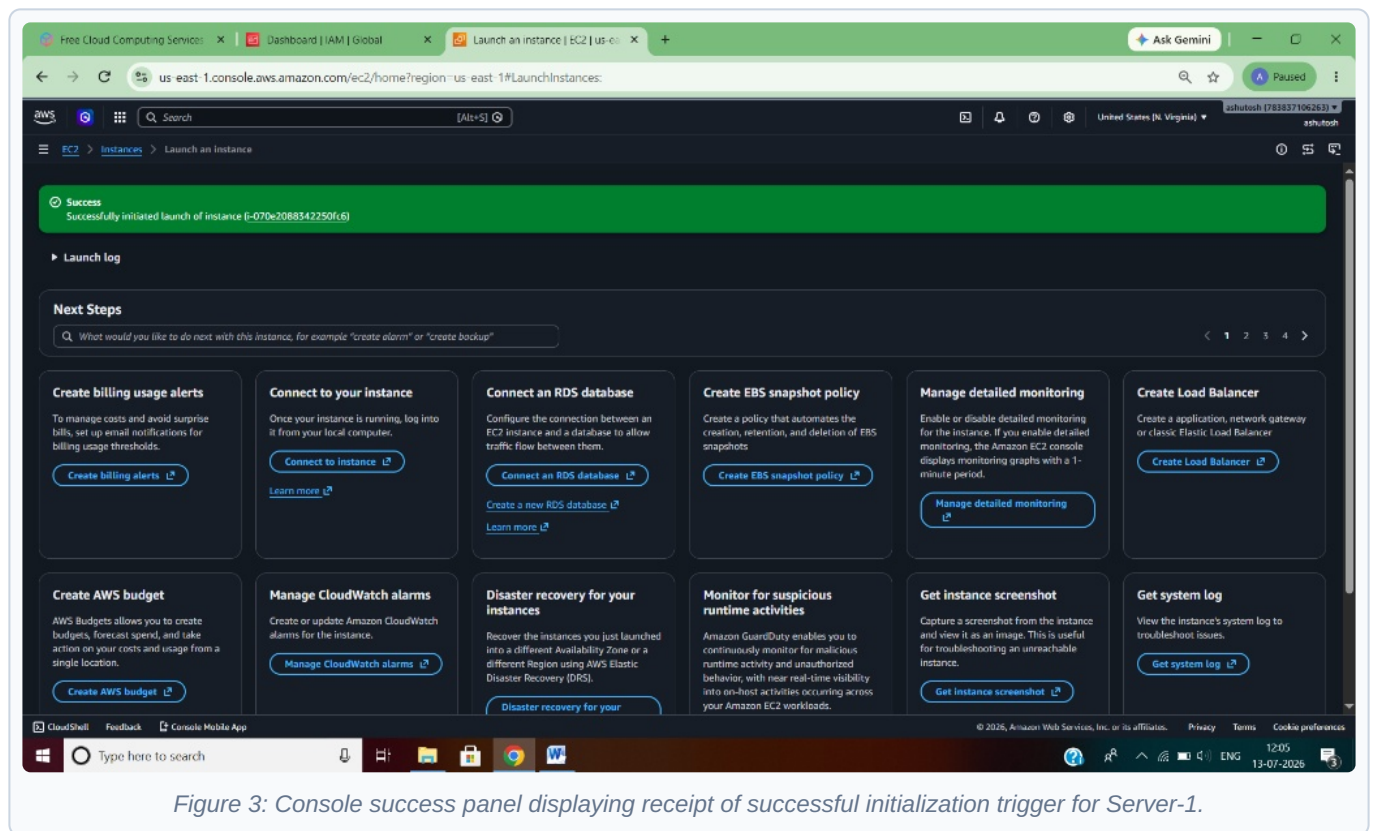


Figure 3: Console success panel displaying receipt of successful initialization trigger for Server-1.

Step 4: Provisioning Redundant Web Endpoint Backend (Server-2)

To provide high availability and load distribution, create a second backend server. Repeat the launch sequence by specifying the identification flag `server-2` while preserving uniform architectural baselines.

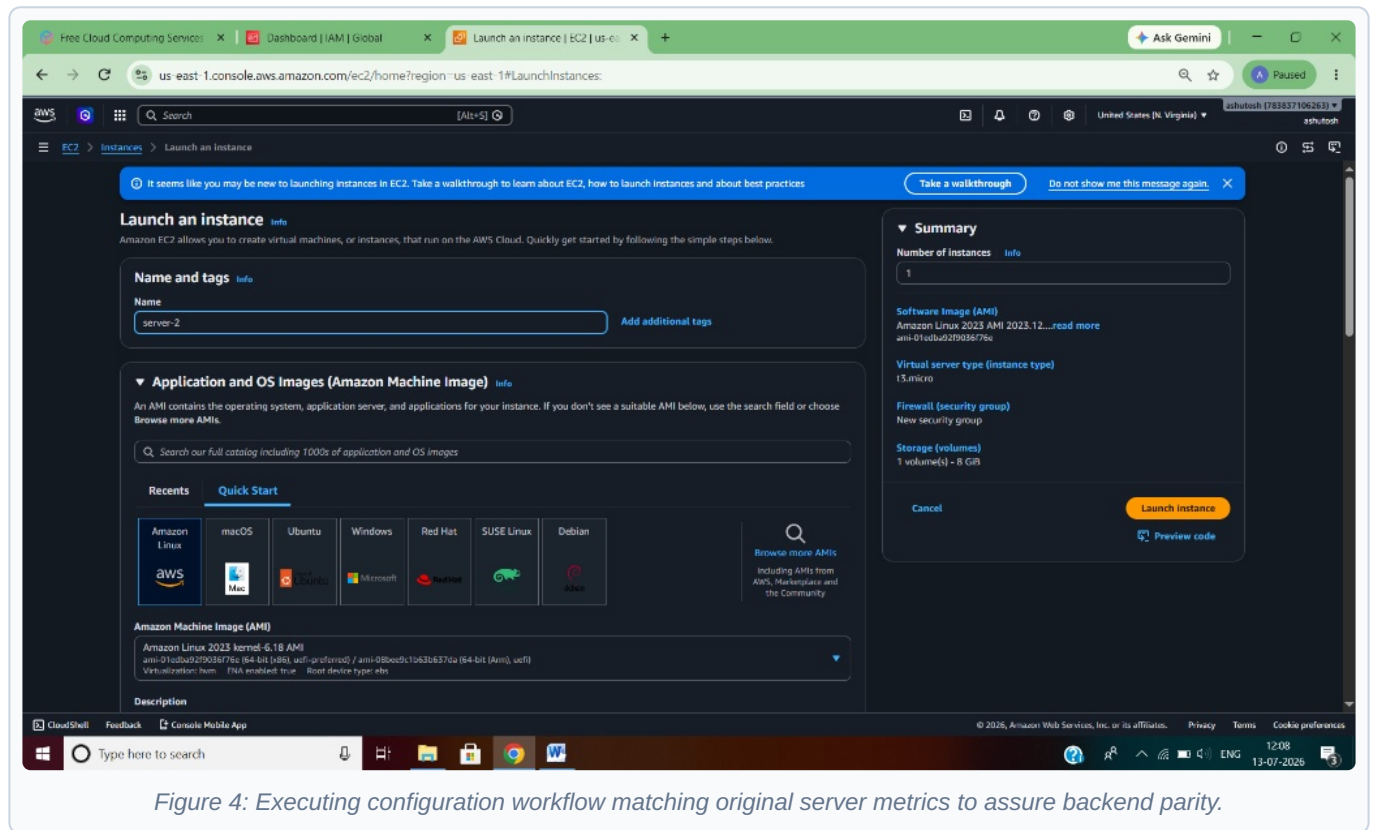
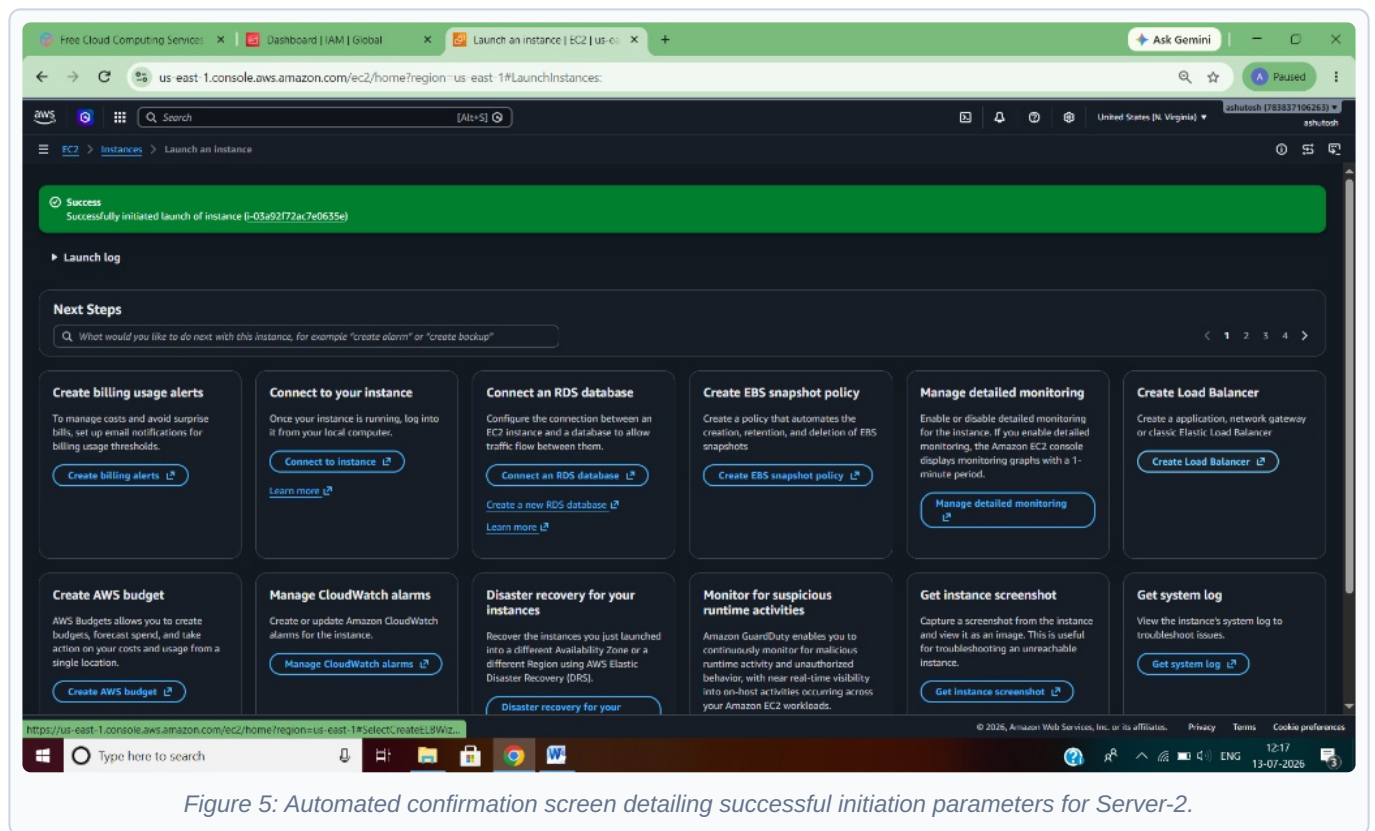


Figure 4: Executing configuration workflow matching original server metrics to assure backend parity.

Step 5: Verification of Server-2 Launch Status

The cloud engine registers the secondary instance. Instance ID *i-03a92f7ac7e0635e* is successfully deployed to active server inventory.



Step 6: Opening the Load Balancer Creation Console

Using the navigation menu, scroll down past network nodes to the dedicated **Load Balancing** tab. Click on **Load Balancers** to load the deployment initialization workspace.

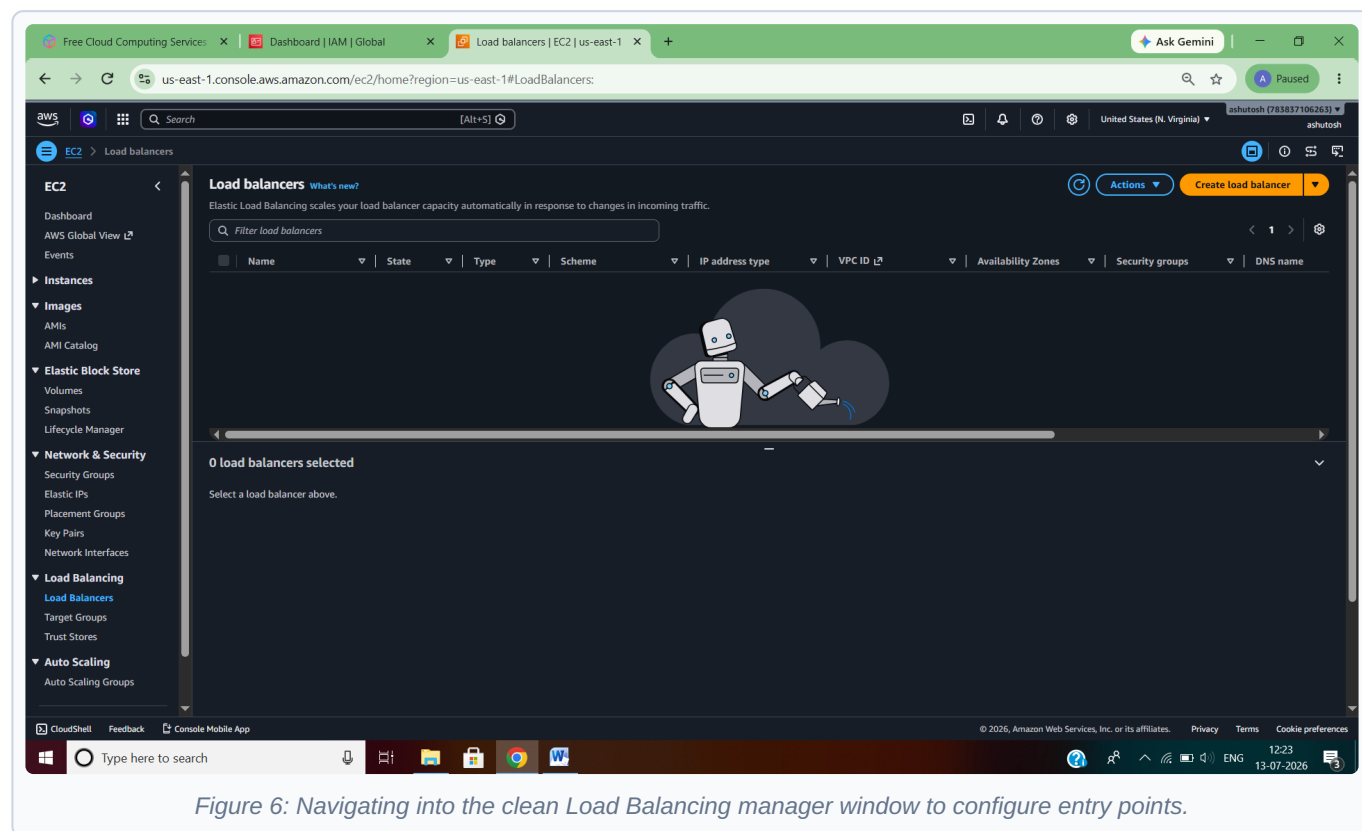


Figure 6: Navigating into the clean Load Balancing manager window to configure entry points.

Step 7: Selecting the Classic Load Balancer Protocol Matrix

The setup wizard presents several specialized options. Navigate to the lower module dedicated to the **Classic Load Balancer (previous generation)** framework which handles basic routing protocols (HTTP, HTTPS, TCP, SSL), and select **Create**.

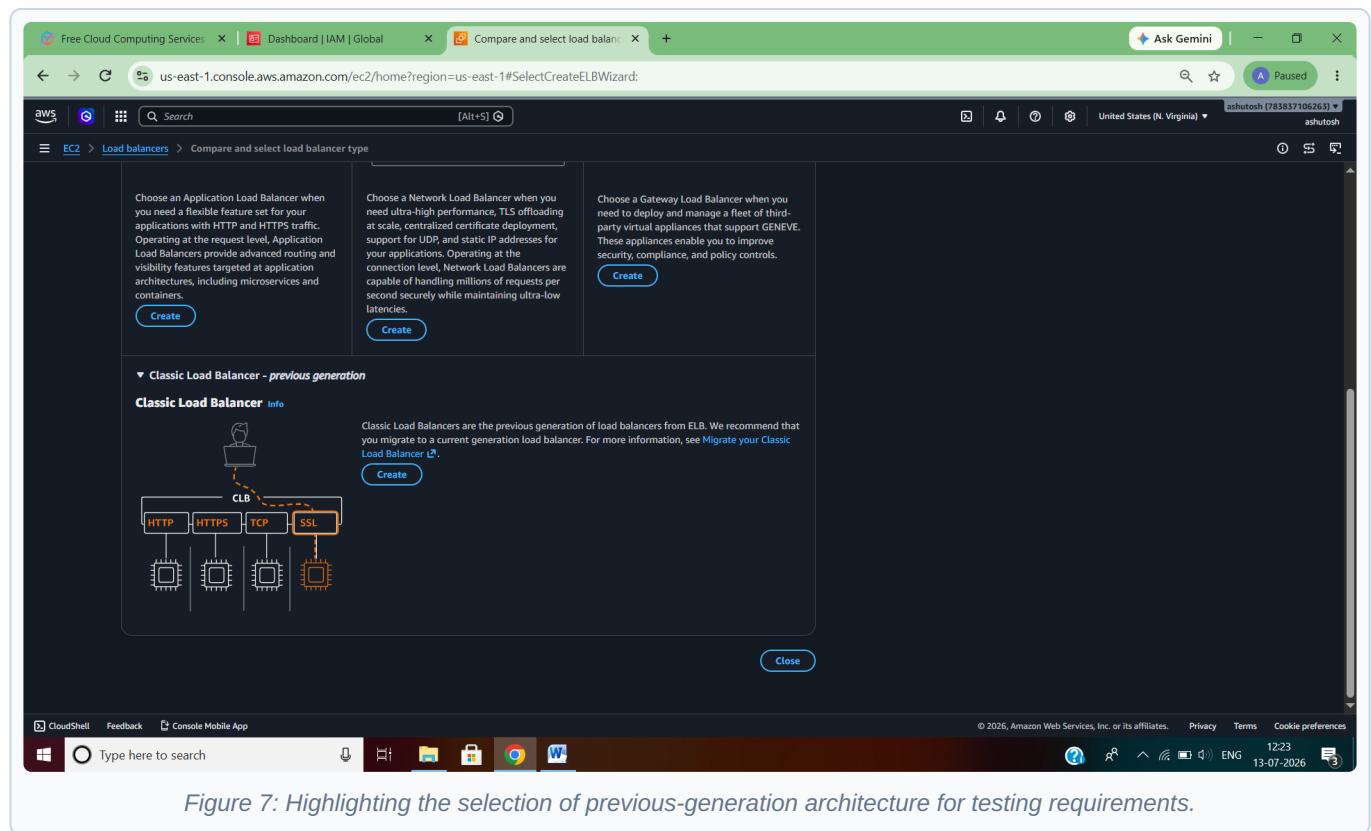


Figure 7: Highlighting the selection of previous-generation architecture for testing requirements.

Step 8: Registering Target Backends to the Balancer

Under the target mapping view, identify active computing workloads. Select the checkboxes corresponding to both active instances: Server-1 and server-2 inside the chosen virtual network topology.

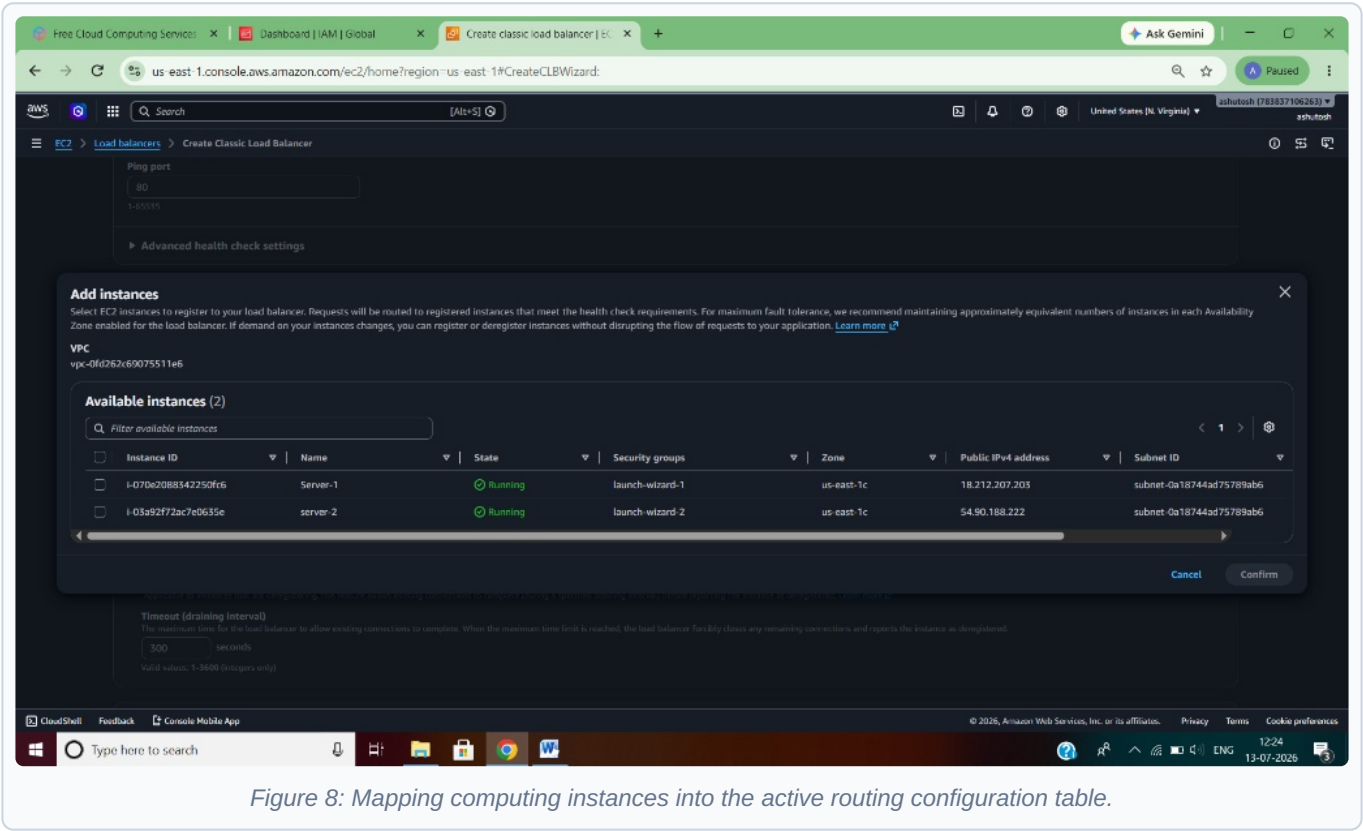


Figure 8: Mapping computing instances into the active routing configuration table.

Step 9: Finalization, Testing & DNS Target Endpoint Generation

The balancer group, designated as lb-1, is successfully provisioned. The infrastructure generates a global Canonical Name (CNAME) address link:

Generated DNS Entry Point Address:

lb-1-1731178461.us-east-1.elb.amazonaws.com

The state flag indicates 0 of 2 instances in service initially. It updates automatically to active operational status as soon as backend listeners report positive health checks.

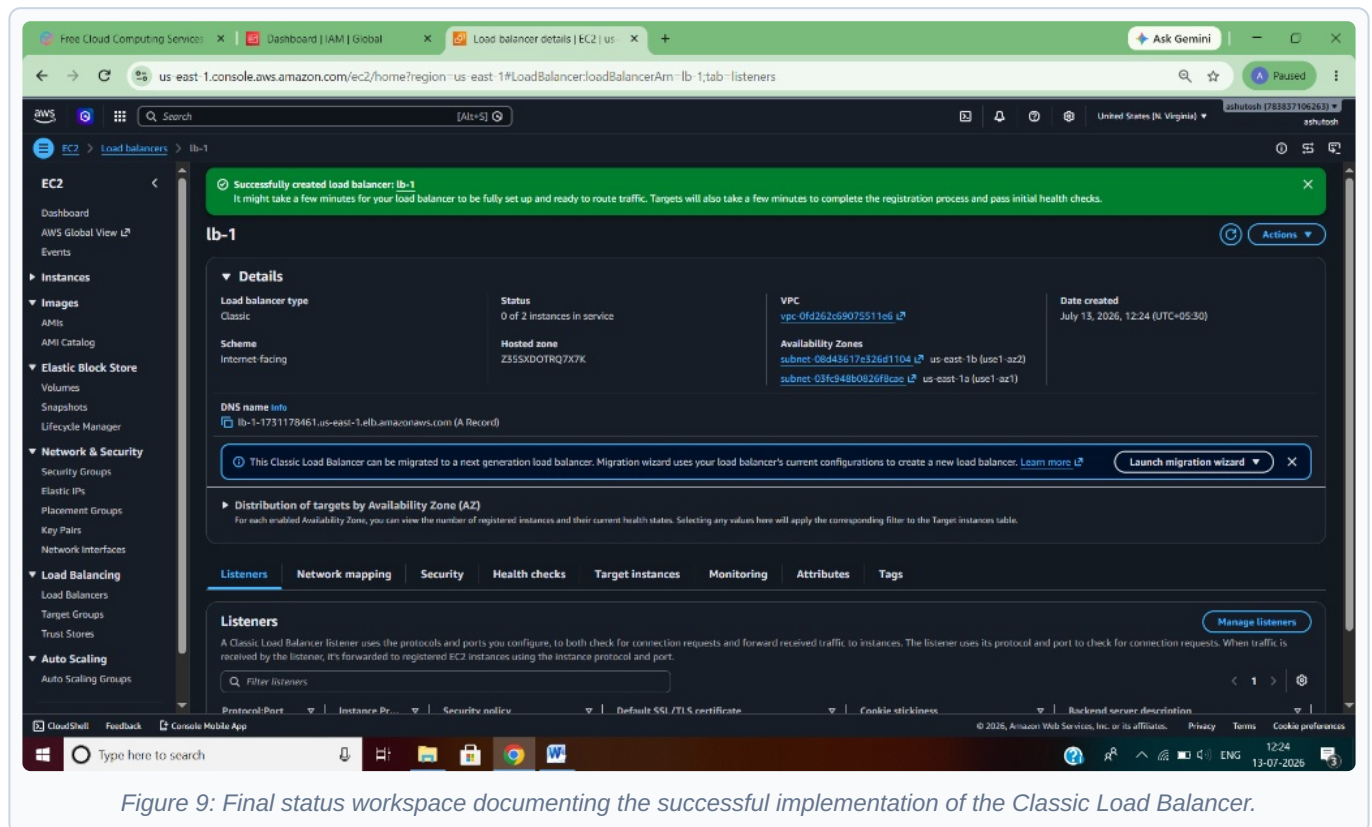


Figure 9: Final status workspace documenting the successful implementation of the Classic Load Balancer.